

Function



CONTENT

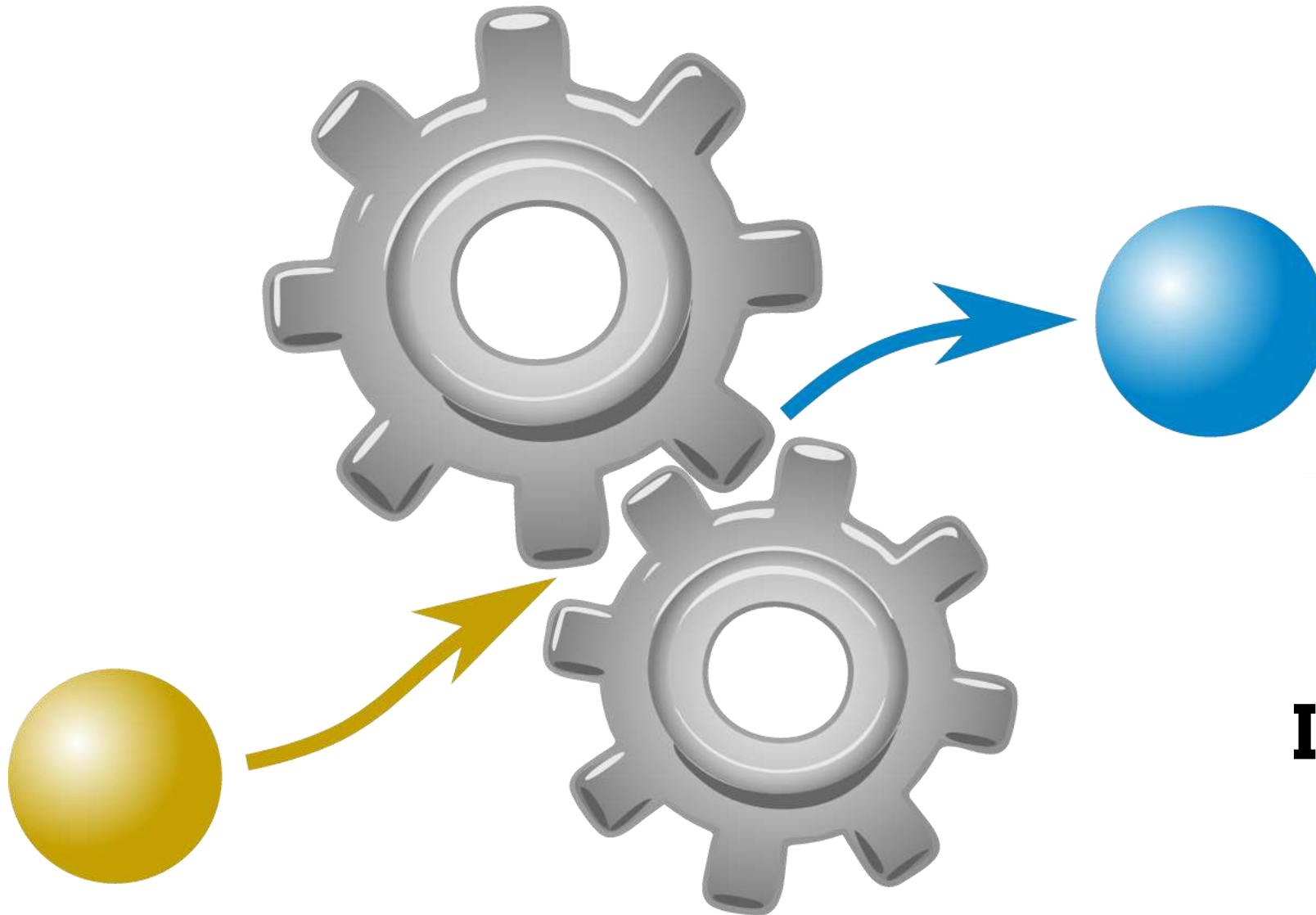
01 Introduction

02 Math Object

03 Number Methods

04 Date & Time





01

Introduction

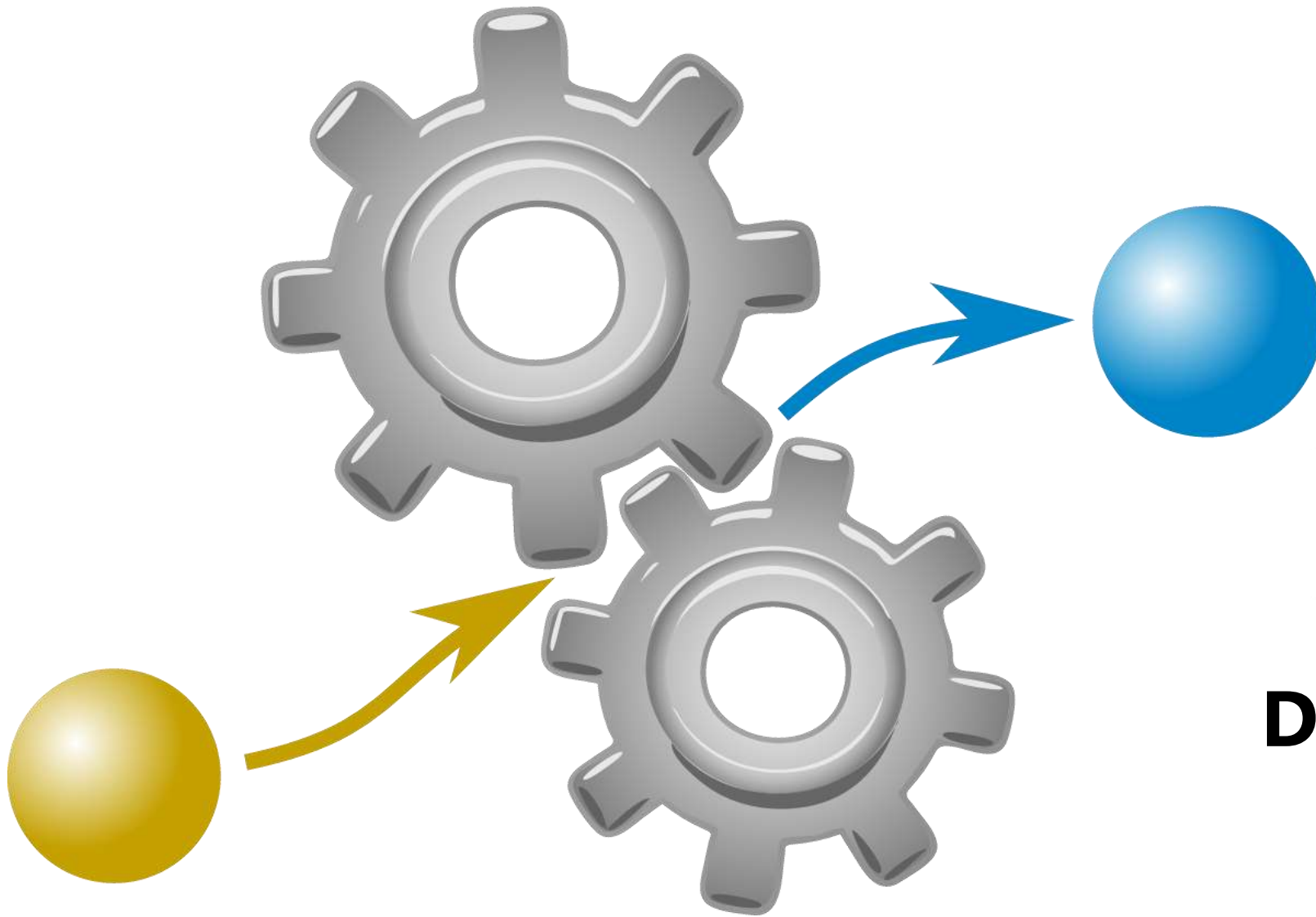
Introduction

A function is a reusable block of code that performs a specific task.

Syntax:

```
function functionName(parameters) {  
    // code to execute  
    return result; // optional  
}
```





02

Declaration

Ways to Declare Functions

Function Declaration :

```
function greet(name) {  
    return "Hello " + name;  
}
```

Function Expression :

```
const greet = function(name) {  
    return "Hello " + name;  
};
```



Key Function Concepts

Parameters vs Arguments :

```
function add(a, b) {           // a, b → parameters
    return a + b;
}
add(2, 3);                     // 2, 3 → arguments
```

Return Value:

```
function double(x) {
    return x * 2;
}
```



Default parameter

A default parameter allows a function to assign a default value to a parameter if no value is passed (or undefined is passed) during the function call.

Syntax:

```
function functionName (parameter = defaultValue) {  
  
    // code  
  
}
```

Example :

```
function greet(name = "Guest") {  
    console.log("Hello, " + name);  
}  
  
greet("Nitish"); // Hello, Nitish  
greet();         // Hello, Guest
```

Default parameters only apply when:

- No argument is passed, or
- The argument is undefined.



Arguments Object

The arguments object is an array-like object available inside regular functions that contains all the arguments passed to that function, regardless of how many parameters are defined

Syntax:

```
function logArgs() {  
  
    console.log(arguments); // array-like object  
}
```

Example :

```
function showArgs() {  
    console.log(arguments);  
    console.log(arguments[0]); // First argument  
    console.log(arguments.length); // Total arguments  
}  
  
showArgs("Hello", "World", 123);
```



Arguments Object

Limitation :

Limitation	Explanation
✗ Not a real array	Can't use array methods like map, filter directly
✗ Not available in arrow functions	Arrow functions do not have their own arguments object
✗ Less readable than rest ...	Modern ES6+ syntax (...args) is preferred

Modern Alternative: Rest Parameter



Rest parameter

Rest Parameters allow a function to accept any number of arguments as an array — beyond the named parameters.

It helps when you don't know how many values will be passed to a function.

Syntax:

```
function functionName(...rest) {  
  
    // rest is an array of arguments  
}
```

The ... (three dots) is the rest operator, and the variable becomes an array.

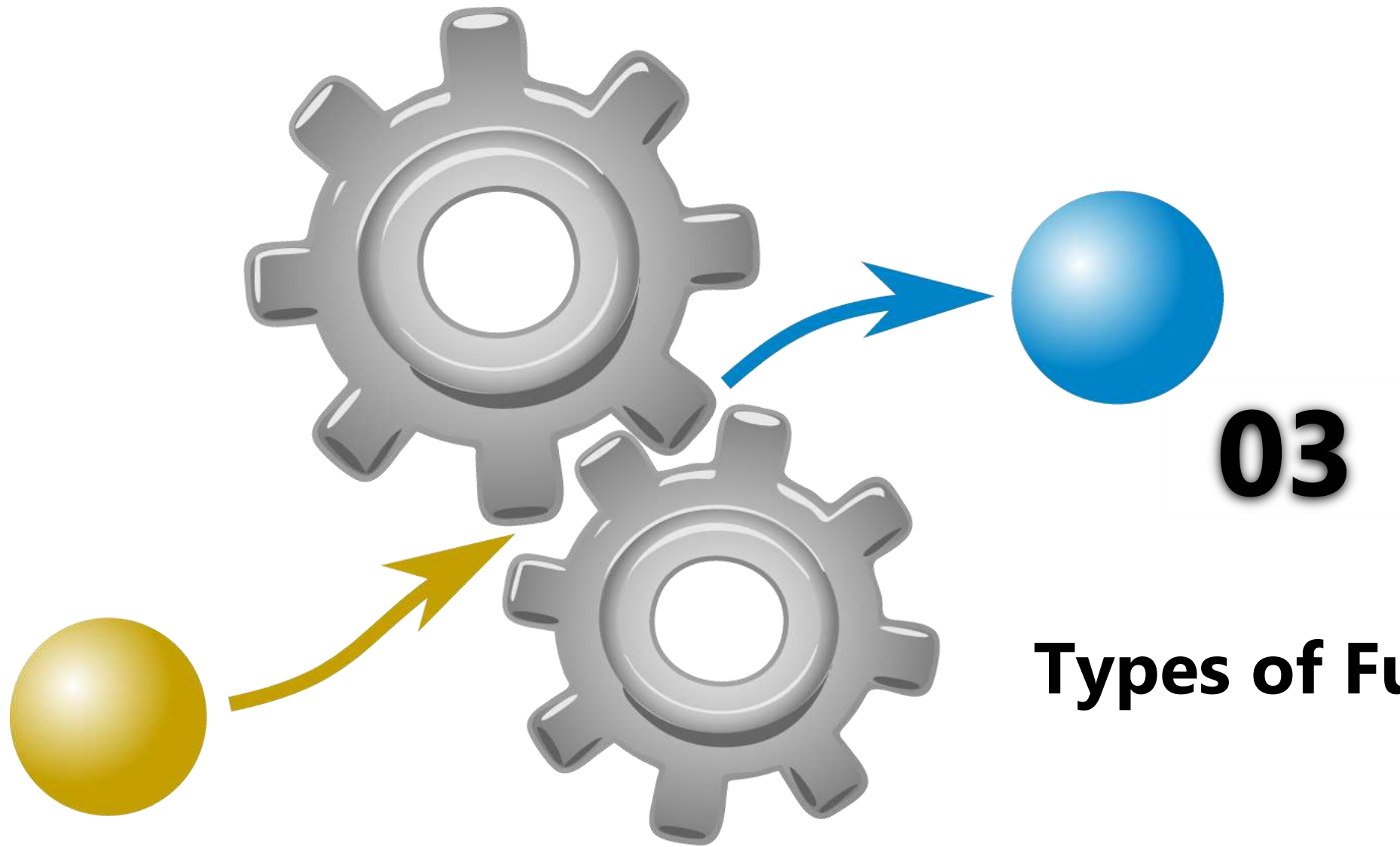
- **Only one rest parameter allowed per function.**
- **It must be the last parameter.**

Example :

```
function sum(...numbers) {  
    return numbers.reduce((total, num) => total + num, 0);  
}
```

```
console.log(sum(1, 2, 3));           // 6  
console.log(sum(5, 10, 20, 15));    // 50
```





Types of Function

Pure Function

A pure function is a function that:
Always returns the same output for the same input.
Has no side effects (it doesn't change or depend on anything outside itself).

Example :

```
function add(a, b) {  
  return a + b;  
}
```

```
console.log(add(2, 3)); // 5  
console.log(add(2, 3)); // 5 (Always same output ✓)
```

- **No global variables used ✓**
- **Doesn't modify anything outside ✓**
- **Returns the same result for same inputs ✓**



Impure Function

**An impure function is a function that:
Does not always return the same output for the same input,
Or causes side effects — meaning it interacts with or modifies something outside its own scope.**

Example :

```
let counter = 0;

function increment() {
  counter++;          // Modifies external variable
  return counter;
}

console.log(increment()); // 1
console.log(increment()); // 2 (✗ Same input, different output)
```



Callback function

A **callback function** is a **function passed as an argument** to another function, and it is **called (or executed)** later inside that function.

Or

A **callback** is like saying:
"When you're done, call me back and run this function."

Example :

```
function greet(name, callback) {  
  console.log("Hello, " + name);  
  callback();           // calling the callback function  
}  
  
function sayBye() {  
  console.log("Goodbye!");  
}  
  
greet("Nitish", sayBye);
```



Callback function

Anonymous Function as Callback :

It's a function that you **define without naming it**, often used when you **don't need to reuse** the function elsewhere.

Example :

```
function calculate(a, b, callback) {  
  let result = a + b;  
  callback(result);  
}
```

```
calculate(5, 7, function (res) {  
  console.log("The result is", res);  
});
```



High-Order Function

A **Higher-Order Function (HOF)** is a function that **either**:

- **Takes another function as an argument, or**
- **Returns a function,**
- **Or does both.**

Function Taking Another Function :

```
function greet(name) {  
  return "Hello " + name;  
}
```

```
function processUser(callback) {  
  let name = "Nitish";  
  return callback(name); // greet is passed as a callback  
}
```

```
console.log(processUser(greet)); // "Hello Nitish"
```



High-Order Function

Function Returning Another Function :

```
function test() {           // Outer function
  return function greet(name) { // Inner function being returned
    return `hello ${name}`;
  }
}

let x = test();           // x now holds the inner function `greet`
console.log(x("Quad")); // Calls greet("Quad") → Output: "hello Quad"
```



Function Scope and Closures

Function Scope :

Variables declared inside a function **can't be accessed from outside**.

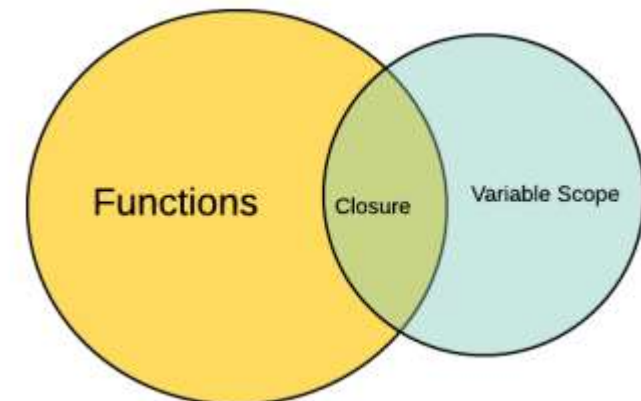
Closure :

A function remembers variables from its **outer scope**, even after the outer function has finished.

Example :

```
function outer() {  
  let count = 0;  
  return function inner() {  
    count++;  
    return count;  
  };  
}
```

```
const counter = outer();  
console.log(counter()); // 1  
console.log(counter()); // 2
```



IIFE

Immediately Invoked Function Expression (IIFE)

An **IIFE (Immediately Invoked Function Expression)** is a function that:
Is **defined** and
Executed immediately after its creation.

Syntax :

```
(function() {  
    // code here runs immediately  
})();
```



Notice It is **wrapped in parentheses** to turn it into an **expression**, and then it is **invoked** with ()

